# AIMO Small Track: Project Manual and Paper Guide

Oct 8, 2026 · @lshraq

## What the project is

We are entered in the **Small Models Track** of the AIMO Interpretability Challenge (NeurIPS 2026). Our current submission, `e27k`, scores **0.86** on the live leaderboard, and the technical report is due **15 November**.

**The task.** Implement `are_robust(model_id, problems) -> list[bool]`. For each (model, olympiad problem) pair, predict whether the model's answer is **robust**, meaning it still gets the problem right after the problem is reworded, or **spurious**, meaning it breaks under them.

**The four models we are scored on:** `Qwen/Qwen3.5-4B`, `Skywork/Skywork-OR1-Math-7B`, `allenai/Olmo-3-7B-Think` and `deepseek-ai/DeepSeek-R1-0528-Qwen3-8B`.

**How a label is made.** The organizers rewrite each problem several ways (rephrase, rename, domain, distract, typos, expert edits) and measure the largest accuracy drop, `max_drop`:

- `max_drop` of 0.10 or less: robust;
- `max_drop` of 0.25 or more: spurious;
- anything in between carries no label and is never scored.

**How we are scored.**

- Accuracy on a private test set of about 30 unpublished AIMO 3 problems, run on Codabench (competition 16180).
- Everything must finish within one hour (3,600 s) on a single RTX PRO 6000 GPU, with no internet.
- Ties are broken by runtime.
- Kaggle is only our free GPU source, not the judge.

| Date | What happens |
|---|---|
| 27 Oct 2026 | Our internal freeze: the submission stops changing |
| 1 Nov 2026, 12:55 UTC | Codabench submissions close |
| 15 Nov 2026 | Technical report and open-source code due |

| Date | What happens |
|---|---|
| 15–30 Nov 2026 | Organizers validate results; teams peer-review up to 3 reports |
| 11–12 Dec 2026 | NeurIPS workshop |

**What we must hand in:** a submission ZIP ( `solution.py` plus a `small.txt` marker), a technical report, and the open-source code that reproduces it. The prize pool is $12,500, and a separate writeup pool rewards generalization, actionable interpretability, negative results and efficient methods.

## Where everything lives

Everything is in one GitHub repository, AIMO-Interpretability-Challenge---Team-BakeNekos, on branch `main` . The small track sits in the folder `AIMO - Interpretability - Small Models track/` , and every path below is relative to it.

| Where | What it holds | When you need it |
|---|---|---|
| `CLAUDE.md` | The working rules: setup, how labels work, Kaggle accounts, coding style | Before changing anything |
| `COMPETITION.md` | The competition rules, task contract and dates | When a rule question comes up |
| `PLAN.md` | The plan; §12 is the handover note and current decisions | To see what is decided |
| `PROGRESS.md` | The narrative log, as numbered Findings 1 to 74 | The main source for the paper |
| `experiments.md` | One entry per experiment run, with configuration and outcome | To look up a run |
| `RESULTS.md` | Every metric table, generated by scripts, never hand-edited | For numbers to cite |
| `REVIEW_GENERALIZATION.md` | The 6 Oct review of every experiment, and corrections to the record | Before writing results |
| `protocols/` | Pre-registrations E34 to E49, written before each run | For the methods section |

| Where | What it holds | When you need it |
|---|---|---|
| `submission/` | Submission builders, and `README.md` with every leaderboard score | For the leaderboard table |
| `datagen/`, `train/` | Feature extraction (GPU) and analysis (CPU) scripts | To reproduce a number |
| `kernels/` | Every Kaggle kernel as pushed, plus each run's record | To reproduce a GPU run |
| `dist/` | The built submission ZIPs, ready to upload | To resubmit |
| `../PAPER_DRAFT/` | The technical report draft; start at its `README.md` | To write the paper |

**What is not in git:** `data/`, `runs/` (downloaded Kaggle outputs), `features/` and `kaggle_runs/` are left out of git, because scripts can rebuild them. `Secrets.env` holds the Kaggle tokens and must never be committed, printed or bundled.

**Kaggle accounts.** Teammates' Kaggle accounts give about 30 GPU-hours a week each, two sessions at a time, on pairs of T4 GPUs.

- `RSD` is read-only: the launchers refuse to push to it.
- `farhanishraqq` is kept out of GPU work.
- Which aliases exist differs per machine; check `Secrets.env` by name only.

## The story so far

In four weeks we went from no data to a 0.86 leaderboard score, and tested about 45 ideas along the way. Most of them failed, and the failures are a big part of what the paper reports.

1. **12–16 Sep: getting set up.** We found six public datasets instead of two, and worked out how the organizers decide "robust". That gave us a training set of 138 labelled examples, all for DeepSeek. We also found two problems in the organizers' own starter code: their example "probe" is really at 50% (a coin flip), and a loading bug removes all spaces from DeepSeek's text.
2. **15–17 Sep: first models.** Problem text alone scored 0.67 on our data. Adding the model's internal signals ("hidden states") raised that to 0.72. Our first real submission, E11 v2, scored **0.64** on the leaderboard.

3. **17–20 Sep: two shortcuts that failed.** We tried making our own labels for the other three models (E15–E19), and checking at test time whether the model's answer survives rewrites (E14). Neither worked.
4. **20 Sep: official labels arrived.** The organizers' training set has only 34 rows for our four models. On those olympiad problems, our model trained on easier school-maths problems did no better than guessing.
5. **22–26 Sep: the winning idea.** We ranked each model's problems by how familiar the text looks to that model, and called the more familiar half robust. Fixing how many to call robust for each model gave `e27k`: **12 of 14 correct (0.86)**, the best score on the board at the time.
6. **24–29 Sep: more ideas, stricter rules.** Each new idea (E28–E38) was written down with its pass/fail test before it ran. All failed their tests; E37 failed only because it was too slow.
7. **30 Sep – 1 Oct: checking the winner.** Tests E39–E48 showed `e27k` is fast, gives the same answer every time, and runs in the official environment. They also showed that much of its score comes from getting the counts right, not the ranking. Decision: ship `e27k`.
8. **6–8 Oct: review and one last test.** A full review of every experiment found some mistakes in our records. We then re-ran the one idea that had worked on outside data (E49). It failed its test on 8 Oct, so `e27k` stays.

## Every experiment in plain words

Only one idea reached the submission: ranking problems by how familiar they look to the model (E27). Every other idea was dropped or closed, and each one is a result the paper can report. "Score" below is AUC unless the row says otherwise: 0.5 means guessing, 1.0 means perfect.

| Experiment | The question | What we found | Outcome |
|---|---|---|---|
| E0, E0b | Can the problem text alone predict robustness? | Partly: 0.67 on school-maths problems, and not just by spotting the source dataset | Became the bar to beat |
| E1, E1b | Do the model's internal signals help? | E1 was spoiled by a bug that removes spaces. In E1b, averaging all layers helped (0.75); picking one layer was luck | Kept, for DeepSeek |

| Experiment | The question | What we found | Outcome |
|---|---|---|---|
| E2, E2f | Does how unsure the model is while reasoning help? | Close to guessing, and slow | Dropped |
| E3, E13 | Do all four models run on Kaggle? | Yes | Done |
| E4 | Are signals during reasoning better than signals from the question? | No: the useful signal is in the question | Dropped |
| E7, E7b | Can we label olympiad problems ourselves? | Yes, but only about 1 problem in 4 is usable, at high cost | Too costly |
| E10, E11 | Text plus internal signals, as a submission | 0.72 on our data, 0.64 on the leaderboard | Kept as the fallback |
| Probes | What is in the hidden 14-case leaderboard set? | Half robust; Qwen and DeepSeek lean robust, Skywork and Olmo lean spurious | Used to set the counts |
| E12 | Can simple signals work without any labels? | They mostly measure problem length | Dropped |
| E14 | Does the answer survive our own rewrites? | No better than guessing, and very slow | Dropped |
| E15–E19 | Make our own labels for the other three models | Labels made, then shown to be wrong in E20 | Dropped |
| E20 | Do our own labels match the official ones? | No; slightly worse than chance | Self-labelling stopped |
| E21 | Does the DeepSeek model work on olympiad problems? | No (0.43) | Dropped |
| E22 | A fair test bench on the official problems | Nothing beats guessing | Standard test from then on |
| E23 | Does familiarity predict robustness? | Not on the official rows | Failed, but led to E27 |
| E24 | Our own labels on olympiad problems | Only 15 usable labels for 15.8 GPU-hours | Dropped |

| Experiment | The question | What we found | Outcome |
|---|---|---|---|
| **E27** | **Rank by familiarity on the leaderboard** | Upside down first (0.29 accuracy); the right way up 0.71; with per-model counts **0.86** | **Shipped as `e27k`** |
| E28, E29 | Better ways to measure familiarity | Tied at 0.71 or did worse | Dropped |
| E30 | The shape of internal signals across layers | Looked good, then flipped on new data | Dropped |
| E31, E32 | How much internal signals move when a question is reworded | Good on official rows, failed on new data | Dropped |
| E34a | New AIME labels from Qwen | Qwen solved only 1 of 52 problems reliably | Dropped |
| E35 | Learn from published robustness labels elsewhere | Did not carry over (0.61) | Dropped |
| E36, E37 | Compare the model's answers at four reasoning lengths | Worked on outside data (0.77, then 0.67), but judged too slow | Closed; retried as E49 |
| E38 | Can one long run replace the four? | No | Dropped |
| E39 | Is `e27k` stable and fast? | Yes: 334 s, same answers every run | Passed |
| E40 | Is 12 of 14 just luck? | The counts explain about one case; the ranking adds about one more | Explained |
| E41 | Has the hidden leaderboard set changed? | No | Freeze decided |
| E42 | Does the ZIP run in the official environment? | Yes | Passed |
| E43 | How much do the counts depend on the test set's balance? | It is a bet on two models, Skywork and Olmo | Known risk |
| E44 | Do the models rank problems differently? | No, almost the same order | For the paper |

| Experiment | The question | What we found | Outcome |
|---|---|---|---|
| E45, E46 | Why does familiarity point the other way on official rows? | Not explained; too few rows to tell | Open question |
| E47 | Could the slow methods ever fit the time limit? | We said no; the review found that too strong | Corrected |
| E48 | How many labels would settle these questions? | 63 to 320; we had 33 to 61 | Explains the many "maybes" |
| E49 | Retest the four-lengths idea, run as a submission would run it | Passed one test (0.66), failed the other (0.52 balanced accuracy) | Failed; `e27k` stays |

## What ships and why

We ship `e27k`, which scored **0.86 (12 of 14)** on the leaderboard. The file is `dist/solution-e27k-tilted-small.zip`, and `python submission/make_e27.py` rebuilds it exactly.

**How `e27k` works, in four steps:**

1. Load the model being asked about.
2. Read each problem once, without letting the model reason or answer.
3. Measure how predictable the problem's wording is to that model. Predictable wording is what we call "familiar".
4. Call the most familiar half of the problems robust. Skywork and Olmo call one fewer, because the leaderboard showed them leaning spurious.

If anything goes wrong, it falls back to a fixed answer per model, so it can never crash the run. It takes about 2 to 3 seconds per problem; 334 seconds covered 144 problems on Kaggle.

**The leaderboard series.** The hidden leaderboard set has only 14 cases, so one case moves the score by 0.07. We uploaded on purpose, one change at a time, to learn something from each score.

| Score | Submission | What it did | What it taught us |
|---|---|---|---|
| **0.86** | **`e27k`** | Familiarity ranking plus per-model counts | The best result; the counts matter |

| Score | Submission | What it did | What it taught us |
|---|---|---|---|
| 0.79 | `e27m` to `e27p` | `e27k` with one count moved up or down | Every change loses a case |
| 0.71 | `e27e` | Familiarity ranking, plain half split | The ranking works the right way up |
| 0.71 | `e27h` | The split for DeepSeek only | All of `e27e`'s gain came from DeepSeek |
| 0.64 | `e27f`, E11 v2 | Best fixed answer per model | The most that fixed answers can score |
| 0.57 | `e27a`, `e27b`, `e27c` | Models trained on the 34 official labels | Too few labels to learn from |
| 0.50 | all-False | Says "spurious" for everything | The set is half robust |
| 0.29 | `e27d` | Familiarity ranking, upside down | Far below chance, so the ranking is real |

**Backups:** `e27e` (0.71) needs no per-model counts. E11 v2 (0.64) needs no GPU at all.

**The honest caveat.** The per-model counts were read off the 14 leaderboard cases. The private test set may be balanced differently, so 0.86 is an upper estimate, not a promise.

## What we learned

The main lesson: with only 34 official labels, almost nothing can be proven either way. The ideas that looked best offline often did worst on the leaderboard. These are the points the paper should carry, with the Finding numbers from `PROGRESS.md`.

1. **The organizers' example method is a coin flip.** Their probe baseline averages 50% per layer. Its 58% headline comes from picking the luckiest result afterwards (Findings 2, 11).
2. **The starter code has a bug.** Loading DeepSeek's text the standard way removes every space (Finding 14).
3. **Robustness belongs to the problem, not the model.** All four models agree on 20 of 20 shared problems (Finding 38), and they rank problems by familiarity in almost the same order (Finding 68).
4. **What works on school maths fails on olympiad problems.** Our best school-maths model scored no better than guessing on olympiad problems (Findings 32, 39). Some signals even flip direction, problem length among them (review, 6 Oct).

5. **Making our own labels did not work.** Our labels disagreed with the official ones, and on hard problems we could barely make any (Findings 31, 48).
6. **Letting the model reason at test time is expensive.** It took 19,000 to 32,000 tokens (word pieces) per problem on Kaggle's GPUs (Finding 74). Whether that fits on the real competition GPU was never measured.
7. **Offline tests and the leaderboard disagreed.** Ideas that passed our offline tests failed on the leaderboard, and our winner fails the offline test (Findings 50, 58).
8. **We never had enough labels.** Settling these questions needs 63 to 320 labelled rows; we had 33 to 61 (Finding 72). A "no" from us often means "not proven", not "false".
9. **Most of the leaderboard gain is in counting right.** Calling the right *number* of problems robust per model explains about one of the two extra cases (Finding 63).
10. **Writing the test down first kept us honest.** From E34 on, every pass/fail test was fixed before the run, and no failed test was later reinterpreted.

## Writing the paper

The paper is mostly written: `PAPER_DRAFT/PAPER.md` has about 9,800 words in 15 sections, with references and four figures. Before **15 November** it needs this week's results added, a few claims corrected, and every number checked against its source table.

### The rules the paper must meet

- **Reproducible from the paper alone.** The organizers must be able to rebuild our score from what the paper says. That is why every number has to come from a script.
- **Open code.** The repository is published alongside the paper.
- **Peer review.** Between 15 and 30 November, each team reviews up to three other teams' papers.
- **Small Models Track only.** The Main Track's work is unfinished, so the paper says nothing about it.

### What the judges reward, and what we have for each

| They reward | What we can show |
|---|---|
| Generalization | Robustness belongs to the problem; school-maths signals fail on olympiad problems; offline tests and the leaderboard disagree |
| Actionable interpretability | A familiarity ranking that needs no training data and a 2 to 3 second read per problem |

| They reward | What we can show |
|---|---|
| Negative results | More than a dozen honest failures, each with the test that ended it |
| Efficiency | `e27k` runs 144 problems in 334 s; reasoning-based methods cost 19,000 to 32,000 tokens per problem |

### What the draft is missing or gets wrong

- [ ] Add E49 (Finding 74) to the failures section (§6), the experiment list (§5), the lessons (§13) and Appendix A.
- [ ] Soften "could not have been submitted at any accuracy" in three places (lines 40, 678 and 905). The truth: too slow on Kaggle's GPUs as we ran it, and never measured on the competition GPU.
- [ ] Add the length finding to §11.1. Familiarity mostly measures problem length, and length points opposite ways on school maths and on olympiad problems.
- [ ] Add the outside-data check to §9: familiarity is close to guessing on MATH-Perturb (0.55) and Putnam (0.53).
- [ ] Say that E24 and E34a sampled differently from the organizers (temperature 0.6 instead of 1.0, shorter limits). Their failures rule out our setup, not self-labelling in general.
- [ ] Check every number against `RESULTS.md`. One mismatch is known: E10 is 0.6916 in the table and 0.7173 in the paper.
- [ ] Fill in co-authors, affiliations and a licence for the repository.
- [ ] Commit the uncommitted edits to `PAPER.md` and `README.md`.
- [ ] Optional: a diagram of the method in §4.

### Suggested schedule

| Dates | What to do |
|---|---|
| 9–15 Oct | Add E49 and the corrections above |
| 16–24 Oct | Check every number against `RESULTS.md`; rebuild the figures |
| 27 Oct | Freeze the submission (`e27k`) |
| 1 Nov | Codabench closes |
| 2–10 Nov | Co-author read-through; final authors and licence |

| Dates | What to do |
|---|---|
| 11–14 Nov | Rebuild everything from a fresh copy of the repository and check it matches |
| 15 Nov | Submit the paper and publish the code |

**Rule of thumb when numbers disagree:** the generated table in `RESULTS.md` is right, and the paper is wrong, because the table came from a script and the paper was typed.

## Commands cheat sheet

Run these from the small-track folder, with the project's own Python: `getting-started\.venv\Scripts\python.exe`, written `python` below. Two exceptions: the paper figures need ordinary system Python, and the Kaggle commands need the system Python that has the Kaggle tool installed.

| To do this | Run this | Notes |
|---|---|---|
| Rebuild every table in `RESULTS.md` | `python scripts/regenerate_results.py` | About 25 minutes, no GPU |
| Check no table would be lost | `python scripts/check_results_coverage.py` | Run after adding a table |
| Rebuild every `e27` submission ZIP | `python submission/make_e27.py` | Same bytes every time |
| Rebuild the five fixed-answer probes | `python submission/make_probes.py` | For re-checking the leaderboard |
| Test a ZIP like Codabench would | `python -m uv run scripts/run_local.py dist/solution-e27k-tilted-small.zip --small` | Run inside `getting-started/` |
| Remake the paper figures | `python PAPER_DRAFT/make_figures.py` | System Python, from the repository root |
| Re-create the review's numbers | `python train/review_generalization.py` | About 3 minutes on CPU the first time |
| See E49's gate error rates | `python train/eval_e49_official.py --simulate-gates` | CPU only |
| Check a Kaggle run | `python scripts/kaggle_followup.py status <job> --account <name>` | Example job: `e49-qwen` |

| To do this | Run this | Notes |
|---|---|---|
| Download a Kaggle run | `python scripts/kaggle_followup.py output <job> --account <name>` | Lands in `runs/` |
| Start a Kaggle run | `python scripts/kaggle_followup.py push <job> --account <name>` | Refuses `RSD`; avoid `farhanishraqq` |
| Before every commit | `python scripts/check_staged_secrets.py` | Stops any token getting into git |

# Glossary

| Word | What it means here |
|---|---|
| Robust | The model still gets the problem right after the problem is reworded |
| Spurious | The model gets the original right, but rewording breaks it |
| Rewording (perturbation) | Changing a problem's wording without changing its answer: rephrasing, renaming, adding a distraction, typos |
| `max_drop` | The biggest fall in accuracy across the organizers' rewordings; it decides the label |
| Unlabelled band | Cases with a drop between 0.10 and 0.25; too close to call, never scored |
| AUC | How well a score ranks robust cases above spurious ones. 0.5 is guessing, 1.0 is perfect |
| Balanced accuracy | Accuracy averaged over the two classes, so always guessing one answer scores 0.5 |
| Leaderboard set | The 14 hidden cases Codabench scores us on before the deadline |
| Private test set | About 30 unseen AIMO 3 problems that decide the prizes after 1 November |
| `train-main-v2` | The organizers' own labelled data: 34 rows for our four models |

| Word | What it means here |
|---|---|
| School maths (MATH) | The easier Hendrycks MATH problems most of our early training data came from |
| Olympiad problems | Competition problems (AIME, AIMO and similar), which is what the test uses |
| Familiarity | How predictable a problem's wording is to the model, measured in one quick read |
| Hidden states | The model's internal numbers while it reads a problem |
| Probe | A small model trained on hidden states to predict a label |
| Tokenizer, token | The tool that cuts text into word pieces (tokens) for the model |
| Budget path (E36, E37, E49) | Asking the model at four reasoning lengths and checking whether its answer stays the same |
| Self-labelling | Making our own robust/spurious labels by running the models ourselves |
| Pre-registration, protocol | Writing down the method and the pass/fail test before running an experiment |
| Gate | A pass/fail test fixed in advance |
| Finding | A numbered result in `PROGRESS.md` (1 to 74) |
| Freeze | The date after which the submission stops changing: 27 October |
| Fallback | A simpler answer used when the main method fails |
| Kaggle, T4 | The free GPU service we borrow compute from, and its GPU type |
| Codabench | The website that runs and scores submissions |